

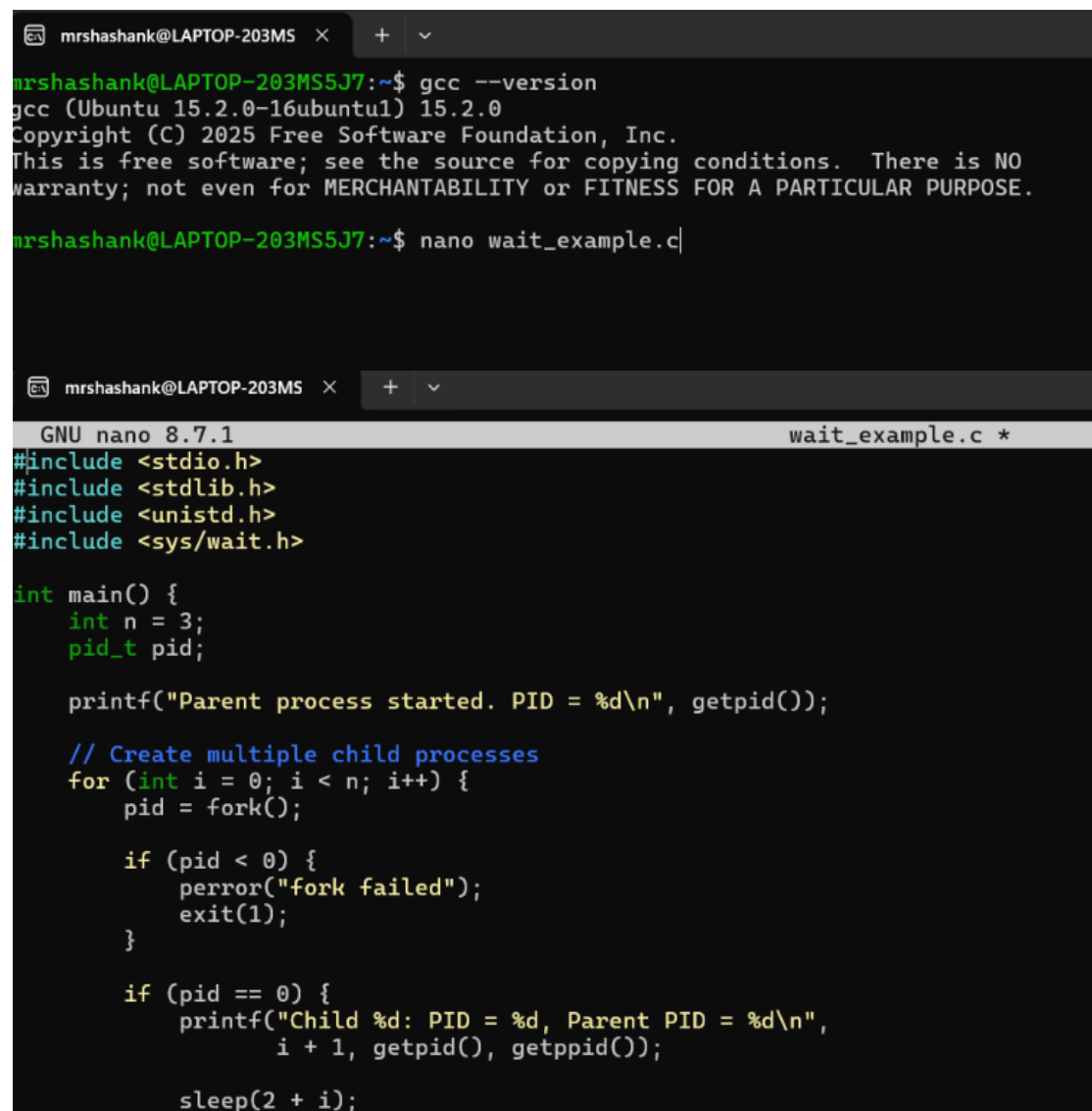
OSSP

SKILL - 4

K. SHASHANK REDDY

2520030215

SEC:08



```
mrshashank@LAPTOP-203MS5J7:~$ gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

mrshashank@LAPTOP-203MS5J7:~$ nano wait_example.c

GNU nano 8.7.1 wait_example.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int n = 3;
    pid_t pid;

    printf("Parent process started. PID = %d\n", getpid());

    // Create multiple child processes
    for (int i = 0; i < n; i++) {
        pid = fork();

        if (pid < 0) {
            perror("fork failed");
            exit(1);
        }

        if (pid == 0) {
            printf("Child %d: PID = %d, Parent PID = %d\n",
                i + 1, getpid(), getppid());

            sleep(2 + i);
        }
    }
}
```

```

        printf("Child %d completed.\n", i + 1);
        exit(0);
    }
}

// Parent waits for all children
for (int i = 0; i < n; i++) {
    pid_t completed = wait(NULL);
    printf("Parent: Child with PID %d has completed.\n", completed);
}

printf("Parent process completed.\n");

return 0;

```

```

mrshashank@LAPTOP-203MS5J7:~$ nano wait_example.c
mrshashank@LAPTOP-203MS5J7:~$ gcc wait_example.c -o wait_example
./wait_example
Parent process started. PID = 659
Child 1: PID = 660, Parent PID = 659
Child 2: PID = 661, Parent PID = 659
Child 3: PID = 662, Parent PID = 659
Child 1 completed.
Parent: Child with PID 660 has completed.
Child 2 completed.
Parent: Child with PID 661 has completed.
Child 3 completed.
Parent: Child with PID 662 has completed.
Parent process completed.
mrshashank@LAPTOP-203MS5J7:~$ |

```

```

GNU nano 8.7.1 waitpid_example
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int n = 3;
    pid_t children[3];

    printf("Parent PID = %d\n", getpid());

    // Create 3 children
    for (int i = 0; i < n; i++) {

        children[i] = fork();

        if (children[i] < 0) {
            perror("fork failed");
            exit(1);
        }

        if (children[i] == 0) {
            printf("Child %d started. PID = %d\n",
                i + 1, getpid());

            sleep(3 - i);

```

```
        printf("Child %d completed. PID = %d\n",
               i + 1, getpid());

        exit(0);
    }
}

// Wait for specific children
for (int i = 0; i < n; i++) {

    int status;

    waitpid(children[i], &status, 0);

    if (WIFEXITED(status)) {
        printf("Parent: Child PID %d completed with exit status %d\n",
               children[i], WEXITSTATUS(status));
    }
}

printf("All children completed.\n");

return 0;
}
```

mrshashank@LAPTOP-203MS5J7:~\$ nano waitpid_example.c
mrshashank@LAPTOP-203MS5J7:~\$ gcc waitpid_example.c -o waitpid_example
mrshashank@LAPTOP-203MS5J7:~\$./waitpid_example
Parent PID = 708
Child 1 started. PID = 709
Child 2 started. PID = 710
Child 3 started. PID = 711
Child 3 completed. PID = 711
Child 2 completed. PID = 710
Child 1 completed. PID = 709
Parent: Child PID 709 completed with exit status 0
Parent: Child PID 710 completed with exit status 0
Parent: Child PID 711 completed with exit status 0
All children completed.
mrshashank@LAPTOP-203MS5J7:~\$ |

```

mrshashank@LAPTOP-203MS  ×  +  ▾
GNU nano 8.7.1 zombie.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0) {

        printf("Child process running. PID = %d\n", getpid());

        printf("Child process exiting...\n");
        exit(0);
    }

    else {

        printf("Parent process PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);

        waitpid(children[i], &status, 0);
        -bash: syntax error near unexpected token `children[i],'
        mrshashank@LAPTOP-203MS5J7:~$ waitpid(pid, &status, 0);
        -bash: syntax error near unexpected token `pid,'
        mrshashank@LAPTOP-203MS5J7:~$ nano zombie.c
        mrshashank@LAPTOP-203MS5J7:~$ gcc zombie.c -o zombie
        mrshashank@LAPTOP-203MS5J7:~$ ./zombie
        Parent process PID = 799
        Child PID = 800
        Parent sleeping for 30 seconds...
        Child process running. PID = 800
        Child process exiting...
        Parent exiting.
        mrshashank@LAPTOP-203MS5J7:~$ |
        mrshashank@LAPTOP-203MS5J7:~$ ps -el | grep zombie
        mrshashank@LAPTOP-203MS5J7:~$ Z 12345 ...
        Z: command not found
        mrshashank@LAPTOP-203MS5J7:~$ ps aux | grep zombie
        mrshash+  850  0.0  0.0  4128  2412 pts/0  S+   04:53   0:00 grep --color=auto zombie
        mrshashank@LAPTOP-203MS5J7:~$ |

```